Informatica

Informatica® Cloud Application Integration
October 2025

Azure OpenAI Connector Guide

Informatica Cloud Application Integration Azure OpenAI Connector Guide
October 2025

© Copyright Informatica LLC 2024, 2025

Publication Date: 2025-11-21

Table of Contents

Preface 4

Chapter 1: Azure OpenAI Connector overview 5

Introduction to Azure OpenAI 5

Chapter 2: Azure OpenAI connections overview 6

Basic connection properties 6

Azure OpenAI connection properties 7

Azure OpenAI connection metadata 7

Azure OpenAI connection actions and objects 8

Using an Azure OpenAI connection in a process 18

Rules and guidelines for using Azure OpenAI connector 19

Preface

Read the *Azure OpenAI Connector Guide* to learn how to set up and use Azure OpenAI connections. This guide assumes that you have an understanding of the Azure OpenAI API concepts.

CHAPTER 1

Azure OpenAI Connector overview

Use Azure OpenAI Connector to integrate APIs described using the OpenAPI specification with Azure services. Azure OpenAI Connector provides seamless interaction between your OpenAPI-defined APIs and various Azure services, enabling powerful integrations and automation.

You can use OpenAI Connector to achieve your general use cases, such as API integration, automated data processing, event-driven architectures, and business process automation.

You can use AzureOpenAI Connector to design workflows and automate tasks, and test and deploy the workflows and applications into production environments.

Azure OpenAI Connector provides the following key features:

- Supports APIs described using OpenAPI version 2.0 and OpenAPI version 3.0.
- Allows you to import OpenAPI definition files to generate connectors.
- Allows integration with other Azure services.
- Provides a low-code environment for integrating APIs and building workflows.
- Provides a visual designer to create workflows and map API operations without manually writing code.
- Manages and configures connectors for different environments, including production and development.

Introduction to Azure OpenAI

Azure OpenAI provides REST API access to OpenAI's language models including GPT-4, GPT-4 Turbo with Vision, GPT-3.5-Turbo, DALLE-3, and embedded model series with the security and enterprise capabilities of Azure.

These models can be easily adapted to your specific tasks, such as content generation, summarization, image understanding, semantic search, and natural language to code translation.

CHAPTER 2

Azure OpenAI connections overview

Use an Azure OpenAI connection for chat completion and file embedding using an endpoint URL and API key.

You can perform various actions, such as chat completion and embedding, and get information about a specific model using a specified ID or a list of available models using the API version.

After you create an Azure OpenAI connection, you can validate, test, and save the connection.

You can then publish the Azure OpenAI connection and click the **Metadata** tab to view the generated process objects for the connection.

Basic connection properties

The following table describes the basic properties that you can configure on the **Properties** tab of the connection creation page:

Property	Description
Name	Required. Unique name for the Azure OpenAI connection that identifies it in the Process Designer. The name must start with an alphabet and can contain only alphabets, numbers, and hyphens (-).
Location	The location of the project or folder where you want to save the connection. Click **Browse** to select a location. If the **Explore** page is currently active and a project or folder is selected, the default location for the connection is the selected project or folder. Otherwise, the default location is the location of the most recently saved asset.
Description	Optional. Description of the connection.
Type	Required. The type of connection you want to use for the connector or service connector. Select **Azure OpenAI**.
Runtime Environment	Required. The runtime environment for the connection. You can run the connection on the Cloud Server, a Secure Agent group, or a Secure Agent machine.

Property	Description
Connection Test	Not supported for Azure OpenAI Connector.
OData-Enabled	Not supported for Azure OpenAI Connector.

Azure OpenAI connection properties

To use the Azure OpenAI API, you need an API key. All API requests must include a valid API key in the `api-key` HTTP header.

The following table defines the Azure OpenAI connection properties that you must configure in the **Connection Properties** section:

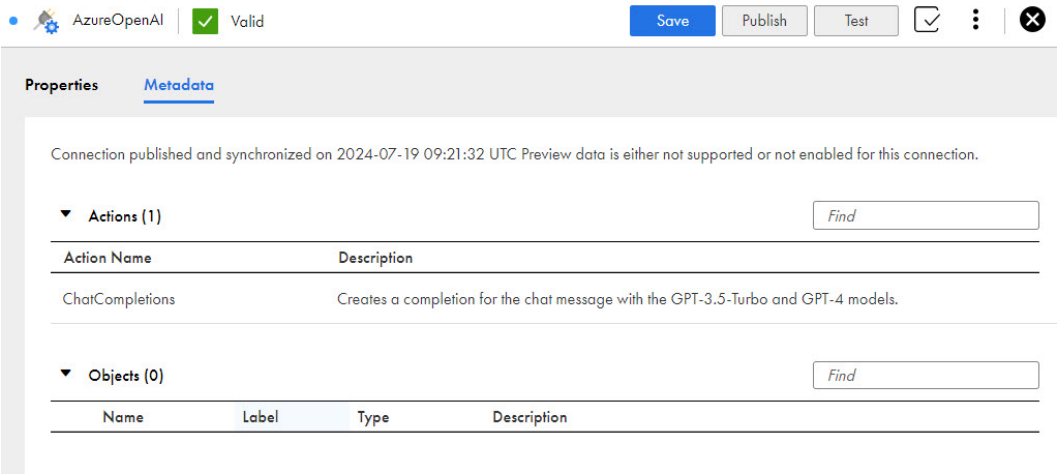
Property	Description
Endpoint_URL	The REST API endpoint for Azure OpenAI. You can find this value in the **Keys & Endpoint** section when examining your resource from the Azure portal. Alternatively, you can find the value in **Azure OpenAI Studio** > **Playground** > **Code View**.
API_Key	The API key to authenticate Azure OpenAI connection requests. You can find this value in the **Keys & Endpoint** section when examining your resource from the Azure portal. Alternatively, you can find the value in **Azure OpenAI Studio** > **Playground** > **Code View**.
API_Version	The service APIs that are versioned using the `api-version` query parameter. All versions follow the `YYYY-MM-DD` date format. For example: `POST https://<RESOURCE_NAME>.openai.azure.com/openai/deployments/<DEPLOYMENT_NAME>/completions?api-version=2024-06-01`

Azure OpenAI connection metadata

After you create an Azure OpenAI connection, you can validate and save the connection. You can then publish the Azure OpenAI connection and click the **Metadata** tab to view the generated process objects for the connection.

When you publish an Azure OpenAI connection in Application Integration, the **Actions** and **Objects** appears on the **Metadata** tab.

The following image shows the **Metadata** tab for a published Azure OpenAI connection:



Azure OpenAI connection actions and objects

When you publish an Azure OpenAI connection in Application Integration, the **Actions** and **Objects** appear on the **Metadata** tab.

Consider the following information when you use specific actions and objects:

- The **ChatCompletions** action creates a completion for the chat message.
  The following input fields are required:
  - chat_completions_request_body - The request body which consists of a series of messages. The model generates a response to the last message, using earlier messages as context.
    The following snippet is a sample request body:

```
<CreateChatCompletionRequest>
    <messages>
        <role>user</role>
        <content>I'm in USA right now</content>
    </messages>
    <messages>
        <role>user</role>
        <content>What is the weather tommorow?</content>
    </messages>
    <temperature>1</temperature>
    <stream>false</stream>
    <stop>some stop</stop>
    <max_tokens>14</max_tokens>
    <presence_penalty>1</presence_penalty>
    <frequency_penalty>1</frequency_penalty>
    <logit_bias/>
    <user>someUser</user>
    <n>1</n>
    <seed>1</seed>
    <logprobs>true</logprobs>
    <top_logprobs>1</top_logprobs>
    <response_format>
        <type>text</type>
    </response_format>
</CreateChatCompletionRequest>
```

  - deployment_id - The name of your model deployment. You must first deploy a model before you can make calls.

- api_version - The API version to use for this operation. This follows the YYYY-MM-DD or YYYY-MM-DD-preview format.

The following snippet is a sample response:

```
<root>
   <created>1758875013</created>
   <prompt_filter_results>
      <content_filter_results>
         <self_harm>
            <severity>safe</severity>
            <filtered>false</filtered>
         </self_harm>
         <jailbreak>
            <filtered>false</filtered>
            <detected>false</detected>
         </jailbreak>
         <hate>
            <severity>safe</severity>
            <filtered>false</filtered>
         </hate>
         <sexual>
            <severity>safe</severity>
            <filtered>false</filtered>
         </sexual>
         <violence>
            <severity>safe</severity>
            <filtered>false</filtered>
         </violence>
      </content_filter_results>
      <prompt_index>0</prompt_index>
   </prompt_filter_results>
   <usage>
      <completion_tokens>14</completion_tokens>
      <prompt_tokens>24</prompt_tokens>
      <completion_tokens_details>
         <accepted_prediction_tokens>0</accepted_prediction_tokens>
         <audio_tokens>0</audio_tokens>
         <rejected_prediction_tokens>0</rejected_prediction_tokens>
         <reasoning_tokens>0</reasoning_tokens>
      </completion_tokens_details>
      <prompt_tokens_details>
         <audio_tokens>0</audio_tokens>
         <cached_tokens>0</cached_tokens>
      </prompt_tokens_details>
      <total_tokens>38</total_tokens>
   </usage>
   <model>gpt-4.1-mini-2025-04-14</model>
   <id>chatcmpl-CJy1l4oJRg7wiYvF5XJt2lNzp3RcF</id>
   <system_fingerprint>fp_4f3d32ad4e</system_fingerprint>
   <choices>
      <content_filter_results>
         <self_harm>
            <severity>safe</severity>
            <filtered>false</filtered>
         </self_harm>
         <hate>
            <severity>safe</severity>
            <filtered>false</filtered>
         </hate>
         <sexual>
            <severity>safe</severity>
            <filtered>false</filtered>
         </sexual>
         <violence>
            <severity>safe</severity>
            <filtered>false</filtered>
         </violence>
      </content_filter_results>
      <finish_reason>length</finish_reason>
```

```
      <index>0</index>
      <message>
         <role>assistant</role>
         <refusal/>
         <annotations/>
         <content>I don't have access to real-time data, including current weather
updates.</content>
      </message>
      <logprobs>
         <refusal/>
         <content>
            <top_logprobs>
               <logprob>-0.00092492</logprob>
               <bytes>73</bytes>
               <token>I</token>
            </top_logprobs>
            <logprob>-0.00092492</logprob>
            <bytes>73</bytes>
            <token>I</token>
         </content>
            ...
            ...
            ...
         <content>
            <top_logprobs>
               <logprob>-0.00000019</logprob>
               <bytes>46</bytes>
               <token>.</token>
            </top_logprobs>
            <logprob>-0.00000019</logprob>
            <bytes>46</bytes>
            <token>.</token>
         </content>
      </logprobs>
   </choices>
   <object>chat.completion</object>
</root>
```

Chat Completions with Function Calling

The following snippet is the sample payload of chat_completions_request_body:

```
<CreateChatCompletionRequest xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <messages>
        <role>user</role>
        <content>My name is John and I'm 30 years old</content>
    </messages>
    <tools m:isArray="true">
        <type>function</type>
        <function>
            <name>get_name_age</name>
            <description>Please provide your name and age</description>
            <parameters>
                <type>object</type>
                <properties>
                    <name>
                        <type>string</type>
                        <description>The name of participant</description>
                    </name>
                    <age>
                        <type>integer</type>
                        <description>The age of participant</description>
                    </age>
                </properties>
                <required m:isArray="true">name</required>
                <required m:isArray="true">age</required>
            </parameters>
        </function>
    </tools>
    <logit_bias>
        <item>
```

```
            <id>5935</id>
            <bias>-100</bias>
        </item>
        <item>
            <id>9653</id>
            <bias>-100</bias>
        </item>
        <item>
            <id>1130</id>
            <bias>-100</bias>
        </item>
    </logit_bias>
    <stop m:isArray="true">test value1</stop>
    <stop m:isArray="true">test value 2</stop>
    <tool_choice>
        <type>function</type>
        <function>
            <name>get_name_age</name>
        </function>
    </tool_choice>
</CreateChatCompletionRequest>
```

The following snippet is the sample response:

```
<root>
   <created>1758880263</created>
   <prompt_filter_results>
      <content_filter_results>
         <self_harm>
            <severity>safe</severity>
            <filtered>false</filtered>
         </self_harm>
         <jailbreak>
            <filtered>false</filtered>
            <detected>false</detected>
         </jailbreak>
         <hate>
            <severity>safe</severity>
            <filtered>false</filtered>
         </hate>
         <violence>
            <severity>safe</severity>
            <filtered>false</filtered>
         </violence>
      </content_filter_results>
      <prompt_index>0</prompt_index>
   </prompt_filter_results>
   <usage>
      <completion_tokens>10</completion_tokens>
      <prompt_tokens>80</prompt_tokens>
      <completion_tokens_details>
         <accepted_prediction_tokens>0</accepted_prediction_tokens>
         <audio_tokens>0</audio_tokens>
         <rejected_prediction_tokens>0</rejected_prediction_tokens>
         <reasoning_tokens>0</reasoning_tokens>
      </completion_tokens_details>
      <prompt_tokens_details>
         <audio_tokens>0</audio_tokens>
         <cached_tokens>0</cached_tokens>
      </prompt_tokens_details>
      <total_tokens>90</total_tokens>
   </usage>
   <model>gpt-4.1-mini-2025-04-14</model>
   <id>chatcmpl-CJzORJKv160B0BdgcfUXQDEbaaC3u</id>
   <system_fingerprint>fp_4f3d32ad4e</system_fingerprint>
   <choices>
      <content_filter_results/>
      <finish_reason>stop</finish_reason>
      <index>0</index>
      <message>
         <role>assistant</role>
```

```
            <refusal/>
            <annotations/>
            <tool_calls>
               <function>
                  <name>get_name_age</name>
                  <arguments>{"name":"John","age":33}</arguments>
               </function>
               <id>call_Ks5cBW0lCsuqJgKJnkG3kb9V</id>
               <type>function</type>
            </tool_calls>
            <content/>
         </message>
         <logprobs/>
      </choices>
      <object>chat.completion</object>
   </root>
```

For more information, see the Azure OpenAI documentation.

- The **Embeddings** action creates a vector representation of an input that machine learning models and algorithms can easily consume.
  The following input fields are required:

  - embeddings_request_body - The request body to obtain the embeddings.
  - deployment_id - The name of your model deployment. You must first deploy a model before you can make calls.
  - api_version - The API version to use for this operation. This follows the YYYY-MM-DD format.

  The following snippet is the sample request body for embeddings_request_body field:

```
<root>
    <input>The food was delicious and the waiter...</input>
    <model>text-embedding-3-large</model>
    <encoding_format>float</encoding_format>
    <dimensions>1536</dimensions>
    <user>Jey</user>
</root>
```

  The following snippet is a sample response:

```
<root>
   <data>
      <index>0</index>
      <embedding>-0.01099226</embedding>
      <embedding>0.00275461</embedding>
      <embedding>-0.0057186</embedding>
      <embedding>-0.008061</embedding>
      <embedding>0.00064367</embedding>
      <embedding>0.01056043</embedding>
            ...
            ...
      <embedding>-0.01368799</embedding>
      <embedding>-0.00563026</embedding>
      <embedding>0.02373806</embedding>
      <embedding>0.00878073</embedding>
      <embedding>0.00617006</embedding>
      <embedding>-0.02632909</embedding>
      <embedding>-0.00401087</embedding>
      <object>embedding</object>
   </data>
   <data>
      <index>1</index>
      <embedding>-0.01123379</embedding>
      <embedding>0.04406882</embedding>
      <embedding>-0.01900215</embedding>
      <embedding>0.00220741</embedding>
      <embedding>0.01660522</embedding>
      <embedding>-0.02373825</embedding>
            ...
```

```
            ...
            ...
         <embedding>-0.00999923</embedding>
         <embedding>0.02431582</embedding>
         <embedding>-0.00141235</embedding>
         <embedding>-0.00057848</embedding>
         <embedding>-0.0273914</embedding>
         <embedding>-0.02011398</embedding>
         <object>embedding</object>
      </data>
      <usage>
         <prompt_tokens>12</prompt_tokens>
         <total_tokens>12</total_tokens>
      </usage>
      <model>text-embedding-3-large</model>
      <object>list</object>
   </root>
```

For more information, see the Azure OpenAI documentation.

- The **GetModel** action returns a single model by the specified ID.
  For example:

  - api_version - 2024-02-15-preview
  - model_id - gpt-4o-canvas-2024-09-25

  The following snippet is the sample response:

```
<root>
   <capabilities>
      <embeddings>false</embeddings>
      <completion>false</completion>
      <inference>true</inference>
      <chat_completion>true</chat_completion>
      <fine_tune>false</fine_tune>
   </capabilities>
   <created_at>1731024000</created_at>
   <lifecycle_status>preview</lifecycle_status>
   <id>gpt-4o-canvas-2024-09-25</id>
   <deprecation>
      <inference>1748736000</inference>
   </deprecation>
   <status>succeeded</status>
   <object>model</object>
</root>
```

- The **GetModelList** returns a list of available models by API version.
  Fox example: api_version - 2024-02-15-preview

  The following snippet is the sample response:

```
<root>
   <data>
      <capabilities>
         <embeddings>false</embeddings>
         <completion>false</completion>
         <inference>true</inference>
         <chat_completion>false</chat_completion>
         <fine_tune>false</fine_tune>
      </capabilities>
      <created_at>1691712000</created_at>
      <lifecycle_status>generally-available</lifecycle_status>
      <id>dall-e-3-3.0</id>
      <deprecation>
         <inference>1769817600</inference>
      </deprecation>
      <status>succeeded</status>
      <object>model</object>
   </data>
            ...
            ...
```

```
            ...
      <data>
         <capabilities>
            <embeddings>false</embeddings>
            <completion>false</completion>
            <inference>true</inference>
            <chat_completion>false</chat_completion>
            <fine_tune>false</fine_tune>
         </capabilities>
         <created_at>1744934400</created_at>
         <lifecycle_status>preview</lifecycle_status>
         <id>gpt-image-1</id>
         <deprecation>
            <inference>1761868800</inference>
         </deprecation>
         <status>succeeded</status>
         <object>model</object>
      </data>
      <object>list</object>
   </root>
```

- The **Create Response** action creates the response object.
  Example of CreateResponseRequest object:

```
<CreateResponseRequest xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
<model>gpt-4.1</model>
<input m:isArray="true">
    <type>message</type>
    <role>user</role>
    <content>Define and explain the concept of catastrophic forgetting?</content>
</input>
<previous_response_id>resp_68d14ea51afc8196aecd8308297ed1ee05b2d33fc583cfec</
previous_response_id>
<temperature>0.7</temperature>
<max_output_tokens>200</max_output_tokens>
</CreateResponseRequest>
```

  The following snippet is the sample response:

```
<root>
   <instructions/>
   <metadata/>
   <max_tool_calls/>
   <reasoning>
      <summary/>
      <effort/>
   </reasoning>
   <usage>
      <input_tokens_details>
         <cached_tokens>0</cached_tokens>
      </input_tokens_details>
      <total_tokens>433</total_tokens>
      <output_tokens>200</output_tokens>
      <input_tokens>233</input_tokens>
      <output_tokens_details>
         <reasoning_tokens>0</reasoning_tokens>
      </output_tokens_details>
   </usage>
   <created_at>1758882238</created_at>
   <safety_identifier/>
   <error/>
   <tools/>
   <content_filters/>
   <output>
      <role>assistant</role>
      <id>msg_68d669bf326c8196943c9b5b5f9febc205b2d33fc583cfec</id>
      <type>message</type>
      <content>
         <annotations/>
         <text>**Catastrophic forgetting** (also called **catastrophic
interference**) is a phenomenon in artificial neural networks where the model rapidly
```

```
loses or overwrites previously learned knowledge when it is trained on new tasks or
data, especially in a sequential manner.

### Explanation

- **Sequential Learning:** When a neural network is trained on one task and then on
another, the learning process for the new task updates the network's weights.
- **Interference:** These updates can interfere with and overwrite the information
stored from previous tasks, leading to a sharp decline in performance on those
earlier tasks.
- **Why it Happens:** Unlike the human brain, which can retain old knowledge while
learning new things, standard neural networks lack mechanisms to preserve previous
learning unless specifically designed to do so.

### Example

Suppose a neural network is trained to recognize handwritten digits (like in the
MNIST dataset). Later, the same network is trained (without access to the old digit
data) to recognize different types of flowers. After training on</text>
         <type>output_text</type>
      </content>
      <status>incomplete</status>
   </output>
   <top_p>1</top_p>
   <previous_response_id>resp_68d14ea51afc8196aecd8308297ed1ee05b2d33fc583cfec</
previous_response_id>
   <temperature>0.7</temperature>
   <tool_choice>auto</tool_choice>
   <model>gpt-4.1</model>
   <service_tier>default</service_tier>
   <id>resp_68d669be7e8c8196bc44149363affd9305b2d33fc583cfec</id>
   <text>
      <format>
         <type>text</type>
      </format>
   </text>
   <incomplete_details>
      <reason>max_output_tokens</reason>
   </incomplete_details>
   <prompt_cache_key/>
   <truncation>disabled</truncation>
   <store>true</store>
   <parallel_tool_calls>true</parallel_tool_calls>
   <background>false</background>
   <user/>
   <object>response</object>
   <status>incomplete</status>
   <max_output_tokens>200</max_output_tokens>
</root>
```

- The **Get Response** action returns a single response object.
  Example of input parameters:

  - api_version - 2025-04-01-preview
  - response_id - resp_68d669be7e8c8196bc44149363affd9305b2d33fc583cfec

  The following snippet is the sample response:

```
<root>
   <instructions/>
   <metadata/>
   <max_tool_calls/>
   <reasoning>
      <summary/>
      <effort/>
   </reasoning>
   <usage>
      <input_tokens_details>
         <cached_tokens>0</cached_tokens>
      </input_tokens_details>
```

```
      <total_tokens>433</total_tokens>
      <output_tokens>200</output_tokens>
      <input_tokens>233</input_tokens>
      <output_tokens_details>
         <reasoning_tokens>0</reasoning_tokens>
      </output_tokens_details>
   </usage>
   <created_at>1758882238</created_at>
   <safety_identifier/>
   <error/>
   <tools/>
   <content_filters/>
   <output>
      <role>assistant</role>
      <id>msg_68d669bf326c8196943c9b5b5f9febc205b2d33fc583cfec</id>
      <type>message</type>
      <content>
         <annotations/>
         <text>**Catastrophic forgetting** (also called **catastrophic
interference**) is a phenomenon in artificial neural networks where the model rapidly
loses or overwrites previously learned knowledge when it is trained on new tasks or
data, especially in a sequential manner.

### Explanation

- **Sequential Learning:** When a neural network is trained on one task and then on
another, the learning process for the new task updates the network's weights.
- **Interference:** These updates can interfere with and overwrite the information
stored from previous tasks, leading to a sharp decline in performance on those
earlier tasks.
- **Why it Happens:** Unlike the human brain, which can retain old knowledge while
learning new things, standard neural networks lack mechanisms to preserve previous
learning unless specifically designed to do so.

### Example

Suppose a neural network is trained to recognize handwritten digits (like in the
MNIST dataset). Later, the same network is trained (without access to the old digit
data) to recognize different types of flowers. After training on</text>
         <type>output_text</type>
      </content>
      <status>incomplete</status>
   </output>
   <top_p>1</top_p>
   <previous_response_id>resp_68d14ea51afc8196aecd8308297ed1ee05b2d33fc583cfec</
previous_response_id>
   <temperature>0.7</temperature>
   <tool_choice>auto</tool_choice>
   <model>gpt-4.1</model>
   <service_tier>default</service_tier>
   <id>resp_68d669be7e8c8196bc44149363affd9305b2d33fc583cfec</id>
   <text>
      <format>
         <type>text</type>
      </format>
   </text>
   <incomplete_details>
      <reason>max_output_tokens</reason>
   </incomplete_details>
   <prompt_cache_key/>
   <truncation>disabled</truncation>
   <store>true</store>
   <parallel_tool_calls>true</parallel_tool_calls>
   <background>false</background>
   <user/>
   <object>response</object>
   <status>incomplete</status>
   <max_output_tokens>200</max_output_tokens>
</root>
```


- The **Get List Input Items** action returns the list of input items for a response.

Example of input parameters:

- api_version - 2025-04-01-preview
- response_id - resp_68d669be7e8c8196bc44149363affd9305b2d33fc583cfec

The following snippet is the sample response:

```
<root>
   <first_id>msg_68d669be80d08196a133aef64b6f767f05b2d33fc583cfec</first_id>
   <data>
      <role>user</role>
      <id>msg_68d669be80d08196a133aef64b6f767f05b2d33fc583cfec</id>
      <type>message</type>
      <content>
         <text>Define and explain the concept of catastrophic forgetting?</text>
         <type>input_text</type>
      </content>
      <status>completed</status>
   </data>
   <data>
      <role>assistant</role>
      <id>msg_68d14ea587588196b23d4e6874d5834105b2d33fc583cfec</id>
      <type>message</type>
      <content>
         <annotations/>
         <text>**Catastrophic forgetting** (also known as **catastrophic
interference**) is a phenomenon in artificial neural networks where the model
**forgets previously learned information upon learning new information**. This
typically occurs when a neural network is trained sequentially on multiple tasks or
data distributions.

### Explanation

- **Sequential Learning Problem:** In standard neural networks, weights are updated
to minimize error on the current task or data. When the network is trained on a new
task, these weights are modified, often overwriting or interfering with the knowledge
acquired from previous tasks.
- **Result:** The network's performance on earlier tasks drops significantly after
training on new tasks, sometimes to the level of random guessing.

### Example

Suppose a neural network is first trained to classify animals (e.g., cats vs. dogs).
Later, it is trained to classify vehicles (e.g., cars vs. trucks) using the same
weights and architecture. After learning vehicles, the network may perform very
poorly on the animal classification task</text>
         <type>output_text</type>
      </content>
      <status>incomplete</status>
   </data>
   <data>
      <role>user</role>
      <id>msg_68d14ea51c748196b42c0e74f88e29eb05b2d33fc583cfec</id>
      <type>message</type>
      <content>
         <text>Define and explain the concept of catastrophic forgetting?</text>
         <type>input_text</type>
      </content>
      <status>completed</status>
   </data>
   <last_id>msg_68d14ea51c748196b42c0e74f88e29eb05b2d33fc583cfec</last_id>
   <has_more>false</has_more>
   <object>list</object>
</root>
```

- The **Delete Response** action deletes a response by specified ID.
  Example of input parameters:

  - api_version - 2025-04-01-preview

- response_id - resp_68d669be7e8c8196bc44149363affd9305b2d33fc583cfec

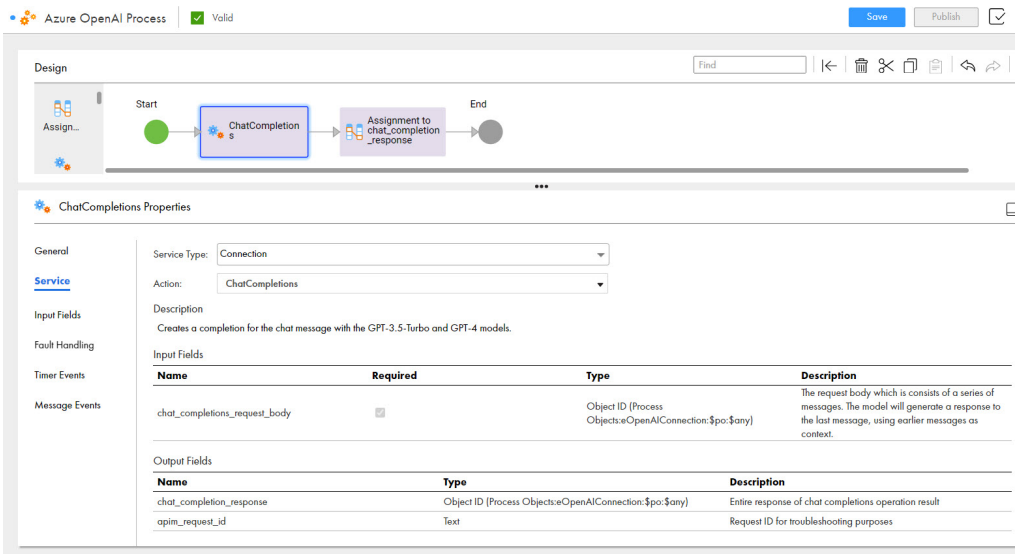
The following snippet is the sample response:

```
<root>
   <deleted>true</deleted>
   <id>resp_68d669be7e8c8196bc44149363affd9305b2d33fc583cfec</id>
   <object>response.deleted</object>
</root>
```

Using an Azure OpenAI connection in a process

After you create an Azure OpenAI connection, you can use it in a Service step in a process.

1. Add a Service step to the process.
2. Click the **Service** tab.
3. From the **Service Type** list, select **Connection**.
4. From the **Connection** list, browse and select the Azure OpenAI connection that you created.
5. From the **Action** list, select the action that you want to perform. For example, select the **ChatCompletions** action to generate a response to the last message using earlier messages for context as shown in the following image:



   In this case, Application Integration populates the **chat_completion_response** and **apim_request_id** objects under the output fields.
6. Configure further steps as needed.
7. Validate, save, and publish the process.

Rules and guidelines for using Azure OpenAI connector

Consider the following rules and guidelines when using the Azure OpenAI connector:

- The Chat completions action enables you to create, define, and use functions as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    ...
    <tools m:isArray="true">
        <type>function</type>
        <function>
            <name>get_name_age</name>
            <description>Please provide your name and age</description>
            <parameters>
                <type>object</type>
                <properties>
                    <name>
                        <type>string</type>
                        <description>The name of participant</description>
                    </name>
                    <age>
                        <type>integer</type>
                        <description>The age of participant</description>
                    </age>
                </properties>
                <required m:isArray="true">name</required>
                <required m:isArray="true">age</required>
            </parameters>
        </function>
    </tools>
    ...
</root>
```

  User defined functions are custom objects. User is responsible for controlling the data types of the function nested objects.

- You can use an image in the Chat Completion action as shown in the following sample code:

```
<root>
    <model>gpt-4-turbo</model>
    <messages>
        <role>user</role>
        <content>
            <type>text</type>
            <text>What's in this image?</text>
        </content>
        <content>
            <type>image_url</type>
            <image_url>
            <url>https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-
wisconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-
boardwalk.jpg</url>
            </image_url>
        </content>
    </messages>
    <max_tokens>300</max_tokens>
</root>
```

- To use fields with data types, such as array, boolean, double, or integer, you must use annotation as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">10.3</field_double>
```

```
    <field_integer m:type="xs:int">5</field_integer>
</root>
```

- Some properties can have different data types. The same variable can be either of type string or array. For example, prompt and stop properties can be string or array. For more information, see the Azure OpenAI documentation.